5章
成長の痛み

マイクロサービスアーキテクチャを採用すると、その過程で難題を経験することになる。そのうちのいくつかは既に見てきたが、事前に警告を得やすくするために、それらをさらに掘り下げていきたい。

この章は、あなたが直面するかもしれない様々な問題について十分な情報を提供することを目的としている。ただ、本書で全ての問題は解決できない。また、この章で説明する問題の多くについては、それらを念頭において書いた前著『マイクロサービスアーキテクチャ』[25]でも、より詳細な内容を扱っている。

また、この章では、旅の途中のどこでこれらの問題が発生しやすいかや、いつこれらの問題に対処が必要なのかを見極めるのに役立つよう、いくつかの兆候を紹介していく。

5.1　サービスが増えれば痛みも増える

マイクロサービスアーキテクチャを採用する中でいつ問題が起こるかには、多くの要因が関係する。サービスの相互作用の複雑さ、組織の規模、サービス数、技術の選択、レイテンシー、稼働時間の要件。これらは痛み、苦しみ、興奮、ストレスをもたらす力のほんの一部だ。つまり、いつ問題に遭遇するかや、実際に問題に遭遇するかどうかをはっきり述べることは難しい。

しかし、10個のサービスを持つ会社と数百のサービスを持つ会社では、問題の種類がかなり異なる傾向にあることは実感している。サービス数は、ある問題がいつ顕在化するかを示す指標としては、何よりも優れているように思う。ここで注意しなければならないのは、私がサービス数について話すときには、特に指定がない限り、論理的に異なるサービスについて話しているということだ。そうした論理的な単位でのサービスは、さらに複数のサービスインスタンスとして本番環境にデプロイされているかもしれない。

マイクロサービス化はスイッチの切り替えとは違う。それはむしろダイヤルを回すようなものだ。ダイヤルを回すとサービスが増え、うまくいけばマイクロサービスの良さを引き出す機会が増える。しかし、ダイヤルを回すたびに、さまざまな問題点にもぶつかることになる。すると、これらの問題を解決する方法を見つける必要が出てくる。そのためには、新しい考え方、新しいスキル、異なる技術、あるいは新しい技術が必要になるだろう。

図5-1は、章の残りで説明する問題点について、サービスが成長していく中のどこで最も起こりやすいかを大まかにマッピングしたものだ。これは非常に非科学的で経験に基づいたものだが、それでも概要としては有用だと考えている。

図5-1　問題点のいくつかがよく現れる場所を高いレベルで示す

これらの問題の全てがこのタイミングに必ず現れると言っているわけではないし、全く現れないとも言っていない。これらの問題には、このような単純な図では説明できないような、いくつかの変数が影響している。その中でも、このような問題に特に影響する要因の一つに、アーキテクチャがどのように結合されているかということがある。より結合されたアーキテクチャでは、堅牢性、テスト、追跡などの問題がより早く顕在化するかもしれない。私がここで期待しているのは、そこにあるかもしれない潜在的な落とし穴に光を当てることだ。

ただし、これはあくまで一般的な指標として捉えてほしい。フィードバックの仕組みを構築しておかなければ、ここで説明する潜在的な指標を実際に得ることはできない。

図の説明はこれくらいにして、これらの問題をもう少し詳しく見ていこう。ここからは、どのような要因がこれらの問題を顕在化させる可能性が高いかと、これらの問題が与える影響、そして実際に顕在化したときに解決するためのポイントをいくつか挙げていきたい。

5.2　所有権のスケール

マイクロサービスアーキテクチャの上で作業する開発者が増えてくると、所有権の扱い方を再考したくなるかもしれない。

Martin Fowlerは以前、一般的なコード所有の観点から所有権の種類を区別したことがある[†1]。私は、それらはマイクロサービスの所有権の文脈でも大まかには機能することを発見した。Martin Fowlerによる所有権の区別は、主にコードの変更を行うという観点からのもので、デプロイや1次サポートを誰が行うかといった観点からの所有権ではない。顕在化する問題について話す前に、まずはMartinが説明している概念を見て、それをマイクロサービスアーキテクチャの文脈に当てはめよう。

[†1] http://bit.ly/2n5pSAf（日本語訳：https://bliki-ja.github.io/CodeOwnership/）

強いコードの所有
全てのサービスには、それぞれ開発者が割り当てられている。他の開発者がサービスを変更したい場合には、その変更をサービスの所有者グループに提出しなければならず、それが許可されるかどうかはサービスの所有者グループが決定する。プルリクエストの使用は、これを処理する方法の一例だ。

弱いコードの所有
全てではないにしても、ほとんどのサービスは開発者に割り当てられている。しかし、プルリクエストのようなものに頼ることなく、誰でもモジュールを直接変更できる。ソース管理上は誰でも自由に変更できるように設定されているが、他の誰かが所有するサービスを変更する場合は、まずその所有者グループに話をすることが期待されている。

コードの共同所有
誰も何も所有しておらず、誰もが何でも変更できる。

5.2.1　この問題はどのようにして現れるか？

サービスや開発者の数が増えてくると、共同所有の問題が出てくるかもしれない。共同所有が機能するには、開発者が相互のつながりを持ち、良い変化がどのようなものか、技術的な観点から特定のサービスがどの方向に進むべきかについて共通の理解を持つ必要がある。

これまでも、マイクロサービスアーキテクチャでこの問題を放置していて、規模が大きくなったときにコードの共同所有が悲惨な結果になるのを見てきた。次に示すのは、あるフィンテック企業の例だ。その企業では、30〜40人の開発者から100人以上の開発者へと、小さなチームが一気に急成長した。しかし、システムの異なる部分に対する責任の割り当てもなく、「何が正しいかはみんなが知っている」という以上の所有権の概念はない状態だった。

そこに現れたのは、システムアーキテクチャの進化に伴う明確なビジョンを持たない、恐ろしく絡み合った「分散モノリス」だった。そこにいた開発者の1人は、自分たちのアーキテクチャを「コランダーアーキテクチャ」と呼んでいた。それは穴だらけで、人々はデータを公開したり、ポイントツーポイントの呼び出しを多く行ったりして、その気になったときにはいつでも「新しい穴を開けてしまう」からだ[†2]。この種の課題はモノリシックシステムでは簡単に解決できる。しかしながら、分散システムでこうした課題を解決するのは、実際にはずっと困難になる。

[†2] コランダーとは穴がたくさん開いたボウルのことで、たとえばパスタの湯切りに使われる。

5.2.2　この問題はいつ発生するか？

小さくスタートした多くのチームにとって、コードの共同所有モデルは理にかなっている。少数の共同開発者（20人程度）なら、このモデルはうまく機能する。開発者の数が増えたり、開発者が分散したりすると、何が良いコミットかや、個々のサービスがどのように進化していくべきなのかといったことについて、全員が同じ理解でいることが難しくなる。

急成長を遂げているチームにとって、共同所有モデルは問題になる。共同所有が機能するには、コンセンサスを形成し、また新しいことが分かったときには

それを更新する時間と空間が必要になるからだ。これは、一般的に人数が増えれば増えるほど難しくなり、新しい人材を急ピッチで雇用している場合（またはプロジェクトに移籍させている場合）には**本当に**難しい。

5.2.3　取り得る解決策

私の経験からすると、複数チームや100人以上の開発者から構成される大規模なマイクロサービスアーキテクチャを実装する組織では、強いコードの所有がほぼ普遍的に採用されている。それによって、良い変更とは何から構成されているのかという基準を各チームで決定しやすくなる。これは、各チームが局所的にコードの共同所有を採用しているともみなせる。このモデルは製品指向のチームにも適用できる。チームがいくつかのサービスを所有していて、それらのサービスがビジネスドメインに基づいて方向付けられている場合、チームはビジネスドメインの1つの領域に集中するようになる。このモデルは、ドメイン知識を構築する顧客中心のチームを維持しやすくし、多くの場合はチーム内のプロダクトオーナーがチームの仕事をガイドする。

5.3　破壊的変更

マイクロサービスは、より広いシステムの一部として存在する。あるサービスは、他のマイクロサービスが提供する機能を使用するか、自分の機能をコンシューマー（他のマイクロサービス）に公開するか、あるいはその両方を行う。マイクロサービスアーキテクチャでは、独立デプロイ可能性の向上を目指しているが、それを実現するには、マイクロサービスに変更を加えたときにコンシューマーを壊さないようにする必要がある。

他のマイクロサービスに公開する機能は**契約**と捉えられる。契約とは、単に「これが私の返すデータです」と言うだけではない。サービスに期待される動作を定義することでもある。コンシューマーとの契約を明確にしたかどうかに関わらず、契約は存在する。サービスに変更を加える際には、この契約を破らないようにする必要がある。そうしないと、厄介な本番環境での問題が発生する可能性があるからだ。

遅かれ早かれ、破壊的変更が引き起こす課題に対処する必要が出てくる。それは、意識的に後方互換性がない変更を行うことを決定したからかもしれないし、あるいは、手元のサービスだけに影響があると思って無邪気に行った変更が、想像していなかったやり方で他のサービスに影響を与えていたからかもしれない。

5.3.1　この問題はどのようにして現れるか？

この問題が顕在化するタイミングで最悪なのは、既存のサービスとの互換性を壊す新しいサービスが本番環境にデプロイされてシステムが稼働停止に陥ったのを目にしたときだ。これは、偶発的な契約の不履行を十分早期にキャッチできていないことを示す兆候だ。こうした問題は、高速にロールバックを行える仕組みを備えていないと、壊滅的なものになる可能性がある。こうした障害モードから得られる唯一のポジティブな点は、たいていはリリース後すぐに問題が顕在化するということだ。ただし、サービス間の契約における滅多に使用されない部分に後方互換性のない変更が加えられた場合はその限りではない。

もう一つの兆候は、複数のサービスを同時にデプロイしようとしているチームが現れ始めた場合だ（これは**ロックステップリリース**と呼ばれることもある）。これは、クライアントとサーバー間の契約変更を管理しようとしているために起こっている兆候かもしれない。チームがたまにしかロックステップリリースを行っていないようなら、それほど悪いことではない。しかし、それが常態化してしまっているようなら、何かしらの調査が必要だ。

5.3.2　この問題はいつ発生するか？

この問題は、特に開発が複数のチームにまたがって行われている場合には、かなり早い段階で遭遇するだろう。変更するサービスとそれを使用するサービスの両方が同じチームによって作業されている場合には、破壊的変更に多少注意が払われる傾向がある。しかし、あるチームがサービスを変更し、他のチームがそれを使用するという状況になると、この問題が頻繁に発生する可能性がでてくる。

時間がたち、それぞれのチームが成熟してくるにつれ、問題がそもそも発生しないように変更を行うようになり、問題を早期にキャッチするための仕組みも整ってくる。

5.3.3　取り得る解決策

契約の違反を管理するためのルールはとてもシンプルだ。

1. 契約を破らないこと。
2. ルール1を参照すること。

大丈夫。これはちょっとした冗談だ。公開している契約を破る変更を行うことは、素晴らしいことではないし、それを管理するのは面倒だ。それを本当にする場合でも、最小限に抑えたいと思うだろう。契約の違反を管理するためのより現実的なルールを次に示そう。

1. 偶発的な破壊的変更を排除する。
2. 破壊的変更を加える前によく考える。どうにかそれを避けられないだろうか。
3. 破壊的変更を行う必要がある場合は、移行期間をコンシューマーに与える。

これらの手順をもう少し詳しく見ていこう。

偶発的な破壊的変更を排除する

マイクロサービスが明示的なスキーマを持つことによって、契約の**構造的な**破壊を素早く検出できる。公開している計算メソッドが、もともとパラメーターとして2つの整数を取っていたのに、1つの整数のみを取るようになったとすれば、それは新しいスキーマから明らかになるはずの破壊的変更だ。このスキーマを開発者に明示することは、この種の問題を早期に発見するのに役立つ。正式なスキーマフォーマットを持っている場合には、もちろんプログラム的にこれを処理するオプションもあるが、そうしたことは私が望むほどには多くは行われていない。プロトコルバッファへの互換性のない変更を禁止してくれるツールである`protolock`[†3]は、そうしたツールの一例だ。

[†3] http://bit.ly/2kUxvbq

多くの人は、スキーマレスの交換フォーマットをまず使用することだろう。この最も一般的な例はJSONだ。理論的にはJSON用に明示的なスキーマを定義することもできる。しかし、実際には使用されていない。開発者が最初のうちは形式的なスキーマの制約を嫌っていたものの、サービス間での破壊的変更に対処しなければならなくなった後で考えを改めるというのは、よくあることだ。また、スキーマを利用しているシリアライズフォーマットの中には、形式的な型によってデータデシリアライズのパフォーマンスが向上しているものもある。このことは注目すべきだろう。

しかし、構造的な破壊は考慮すべき破壊の一部に過ぎない。私たちは**意味的な**破壊も考慮しなければならない。計算メソッドがまだ2つの整数を取るものの、これまでそれらを乗算していたマイクロサービスが加算するようになったなら、それも契約違反だ。実際には、テストを行うことがこれを検出するための最良の方法の一つだ。これについてはすぐ後で触れる。

何をするにしても、開発者が外部契約に変更を加える際には、それを可能な限り明らかにすることが一番の近道だ。これは、データを魔法のようにシリアライズしたり、コードからスキーマを生成したりする技術を避け、代わりにこれらのことを手作業で行うことを意味するかもしれない。サービス契約の変更を困難にすることは、コンシューマーを絶えず壊すことよりも良いことなのだということを信じてほしい。

破壊的変更を加える前によく考える

可能であれば、契約を拡張するような変更を選ぼう。古いものを削除することなく新しい機能をサポートするメソッド、リソース、トピックなどを追加しよう。古いものをサポートしつつ、新しいものをサポートする方法を見つけるようにしてほしい。それによって古いコードをサポートしなければならないかもしれないが、それでも破壊的変更を処理するよりは作業が少なくて済むかもしれない。もし契約を破棄することになった場合、その影響を処理するのは自分自身であることを忘れないでおこう。

移行期間をコンシューマーに与える

最初から明確にしているように、マイクロサービスは**独立してデプロイが可能**なように設計されている。マイクロサービスに変更を加える場合、他の何かをデプロイすることなく、そのマイクロサービスを本番環境にデプロイできるようにする必要がある。そのためには、既存の利用者に影響を与えないような方法でサービス契約を変更する必要がある。これは、新しい契約が利用できるようになっていても、古い契約を利用できるようにすることを意味する。その上で、全てのコンシューマーに、新しいバージョンのサービスに移行するための変更を行う時間を与える必要がある。

これには2つの方法がある。1つ目の方法は、図5-2に示すように、マイクロサービスの2つのバージョンを実行することだ。通知サービスの2つのビルドを同時に利用可能にし、それぞれが異なる互換性のないエンドポイントを公開し、コンシューマーがそれを選択できるようにする。このアプローチの主な課題

は、追加のサービスを実行するためにインフラストラクチャを増やさなければならないこと、サービスのバージョン間でデータの互換性を維持しなければならないこと、実行中の全てのバージョンに対してバグフィックスを行う必要があること、必然的にソースコードの分岐を必要とすることがある。これらの問題は、2つのバージョンを短期間だけ共存させる場合であれば多少緩和される。私がこのアプローチを検討するのは、唯一その状況のときだけだ。

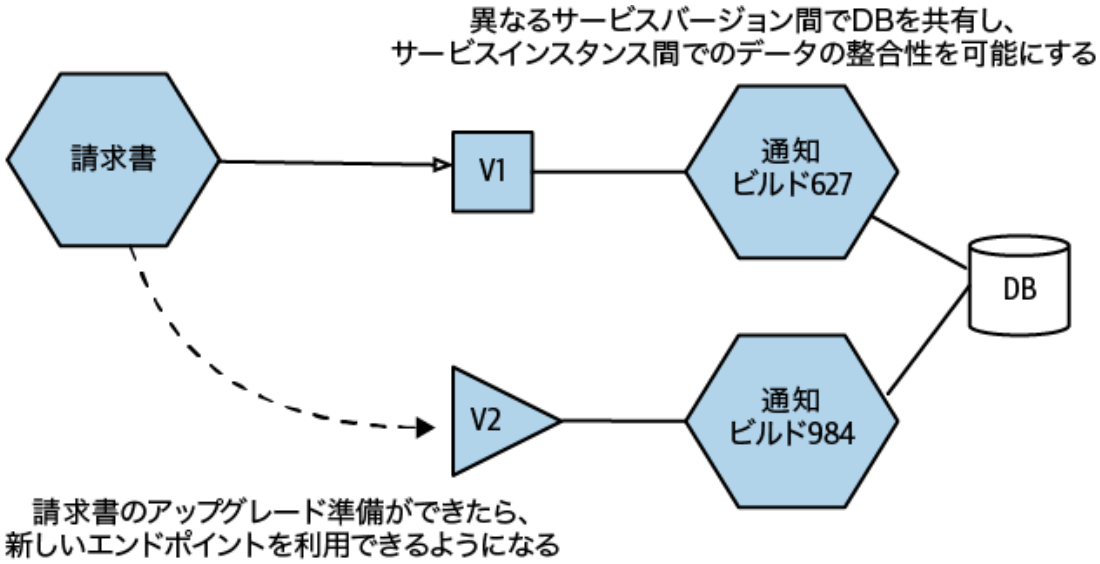

図5-2　後方互換性のない変更をサポートするために、同じマイクロサービスの2つのバージョンを共存させる

私が好むアプローチは、図5-3に示すように、稼働しているマイクロサービスのバージョンが両方の契約をサポートする方法だ。これには、たとえば、2つのAPIを異なるポートで公開することが含まれるかもしれない。このやり方は、マイクロサービスの実装を複雑にするものの、先ほどのアプローチの課題を回避できる。私は過去に、外部のコンシューマーが変更できないことで、数年たっても同じサービスで3つ以上の古い契約をサポートしているチームと話をしたことがある。それは決して楽しい立場ではないが、変更しようとしないコンシューマーがいる場合には最良の選択肢だろう。

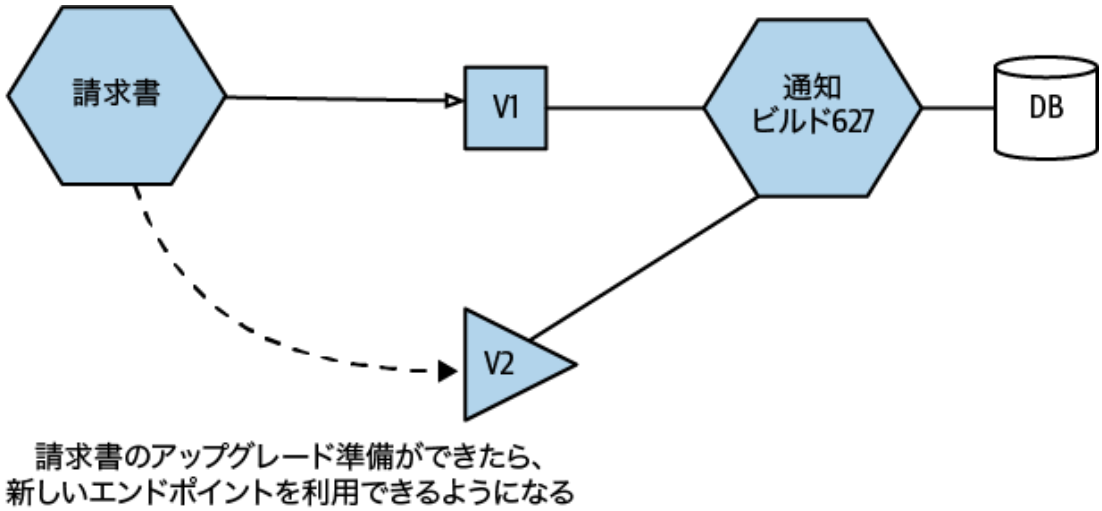

図5-3　1つのサービスで2つの契約を公開する

もちろん、同じチームがコンシューマーとプロデューサーの両方を担当していれば、ロックステップリリースを行い、両方の新しいバージョンを同時にデプロイすることも可能だ。これは頻繁にやりたいことではないが、チーム内であればリリース調整を管理するのは簡単だろう。ただし、くれぐれもこれを習慣化してはいけない。

チーム内での変更は、考慮しなければいけない状況の両側をコントロールできるので、管理しやすくなる。変更したいマイクロサービスがより広く利用されるようになると、変更を管理するためのコストが大きくなる。その結果、チーム内で破壊的変更が気楽な一方で、第三者の組織に公開しているAPIを壊すのは、かなりの苦痛を伴う可能性がある。

どのようにするにしても、あなたのサービスを利用するサービスを管理している人たちとの良好なコミュニケーションが必要だ。何にしても迷惑をかける可能性が高いのだから、仲良くしておくに越したことはないだろう。サービスを利用するコンシューマーを顧客のように扱おう。

この問題に早くから取り組もう

マイクロサービスの数を増やしていく中で、組織は最終的には偶発的な変更をほとんど排除する方法を見つけ出し、そして意図的な変更を処理するための管理された仕組みを考え出すようになる。そうしないと、その影響は非常に大きくなり、マイクロサービスのアーキテクチャでは対応できなくなってしまう。別の言い方をすれば、この問題を解決しない小規模なマイクロサービス組織

は、大規模なマイクロサービス組織になるまでには長続きしないのではないかと私は強く疑っている。

5.4　レポーティング

モノリシックなシステムは、通常モノリシックなデータベースを持っている。そして、モノリシックなデータベース上で全てのデータをまとめて分析するにはデータ間の大がかりな結合操作が伴うことが多く、そのため、ステークホルダーは、一般にレポーティング用のスキーマを所有している。図5-4に示すように、レポーティングは通常、モノリシックなデータベースもしくは読み取り用のレプリカに対して行われる。マイクロサービスアーキテクチャでは、このモノリシックスキーマを分解している。これは、全てのデータにまたがってレポーティングを行う必要性がなくなったということではない。むしろ、データが複数の論理的に分離されたスキーマに分散しているため、より困難になったということを意味している。

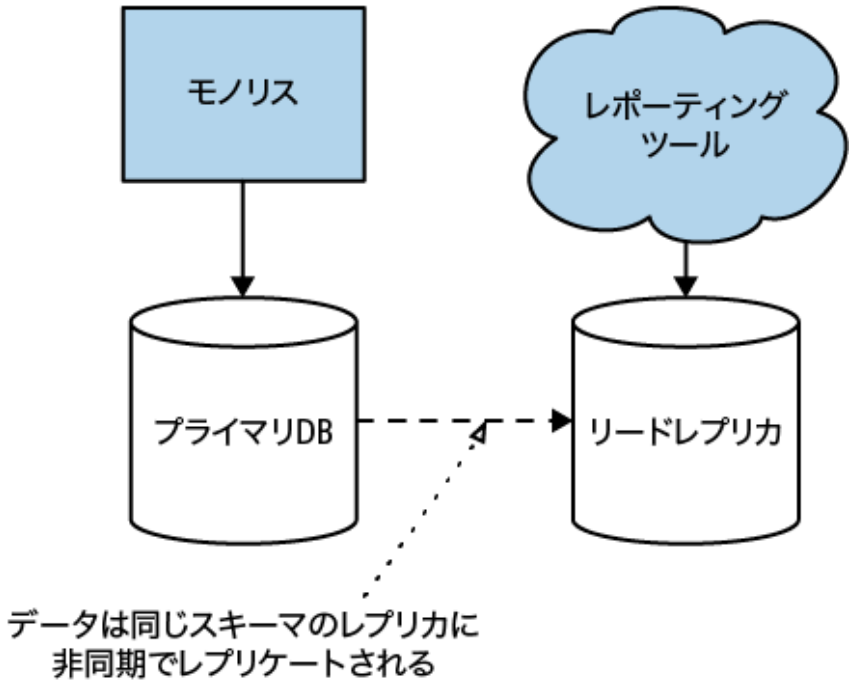

図5-4　モノリスのデータベース上で直接レポーティングが行われている場合

5.4.1　この問題はどのようにして現れるか？

この問題はかなり早い段階で発生する傾向があり、通常、モノリシックなスキーマの分解を考え始めた段階で発生する。できればこれが問題になる前に発見されることを願っているものの、アーキテクチャの方向性がレポーティングに興味を持つステークホルダーにとって不幸をもたらすことに途中まで気付けな

かったプロジェクトを、私は過去に何度も見てきた。レポーティングのニーズは、通常のソフトウェア開発やシステムメンテナンスの領域外で発生することが多いため、早い段階では十分に考慮されていないことが多々ある。

モノリスが既にデータウェアハウスやデータレイクなど、レポーティング目的で専用のデータソースを使用している場合は、この問題を回避できるかもしれない。その場合に必要なのは、マイクロサービスが既存のデータソースに適切なデータをコピーできるようにすることだけだ。

5.4.2　取り得る解決策

1箇所で全てのデータにアクセスできることを気にしているようなステークホルダーは、多くの状況で、SQLを使ってデータベースに直接アクセスできることを期待するツールチェーンやプロセスにも投資していることだろう。また、レポーティングはモノリシックデータベースのスキーマ設計に縛られている可能性も高い。これは、動作方法を変更しない限り、レポーティング用に単一のデータベースを提示する必要があることを意味し、変更の影響を制限するために古いスキーマ設計と一致するものを提示する必要があることを意味する。

この問題を解決するための最も簡単なアプローチは、まず、図5-5で示すように、レポーティング目的でデータを保存するための単一のデータベースへの必要性を、マイクロサービスがデータを保存したり取得したりするために使用するデータベースから切り離すことだ。これにより、レポーティング用データベースの中身と設計を、各サービスのデータストレージ要件の設計と進化から切り離せる。また、この新しいレポーティング用データベースは、レポーティングを行うユーザーの特定要件を念頭に置いて変更できる。あとは、どのようにしたらマイクロサービスが新しいスキーマにデータを「プッシュ」できるかを考え出すだけだ。

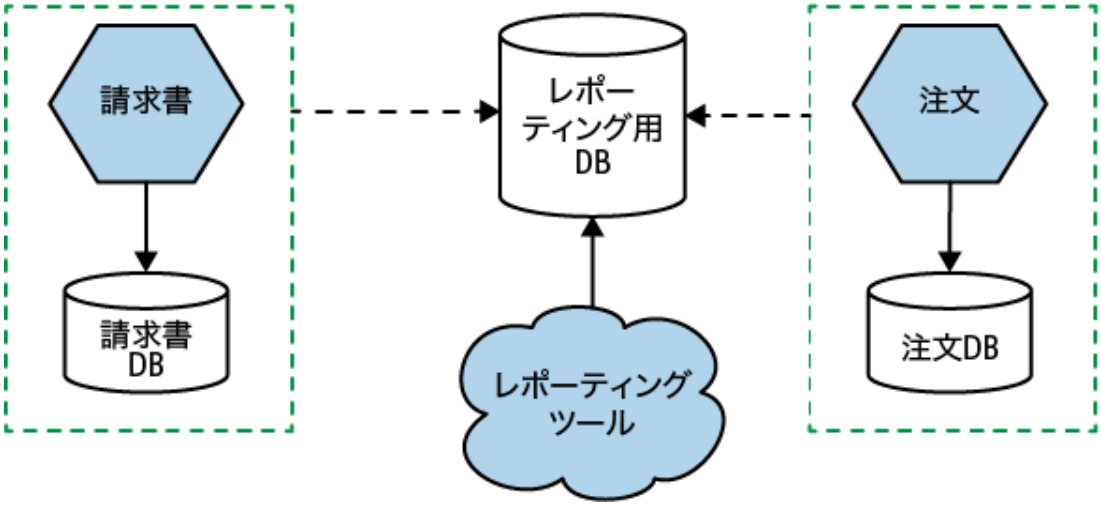

図5-5　異なるマイクロサービスからデータがプッシュされる専用のレポーティング用データベース

この問題に対して取り得る解決策は、既に3章と4章で見てきた。「パターン：変更データキャプチャ」がそうだ。また、複数のマイクロサービスのデータベースが公開するビューから単一のレポーティング用スキーマを提示できるかもしれないため、「パターン：データベースビュー」のようなテクニックも有用だろう。また、レポーティング用のスキーマにデータをコピーするコードをマイクロサービスの一部に組み込んだり、上流サービスのイベントを待ち受けてレポーティング用データベースを生成する中間コンポーネントを持ったりするなど、他のテクニックの使用も検討できるかもしれない。

このトピックに関する課題と取り得る解決策については、『マイクロサービスアーキテクチャ』[25]の5章でさらに詳しく説明している。

5.5　監視とトラブルシューティング

> *モノリスをマイクロサービスに置き換えたことで、全ての停止がミステリー小説の殺人事件のようになった。*
>
> *Honest Status Page（@honest_update）http://bit.ly/2mldxqH*

標準的なモノリシックアプリケーションでは、監視に関してかなり単純なアプローチをとれる。心配するマシンの数が少なく、アプリケーションの障害モードは二値的で、アプリケーションは起動しているか停止しているかのどちらかとなる。マイクロサービスアーキテクチャでは、1つのサービスインスタンスだけの障害やある種類のインスタンスだけの障害が考えられる。それらを適切に拾えるだろうか？

モノリシックシステムでは、CPU使用率が100%の状態が長時間続くと、それが大きな問題であることが分かる。数十、数百のプロセスを持つマイクロサービスアーキテクチャでも、同じことが言えるだろうか？　たった1つのプロセスが100%のCPU使用率で止まっているときに、午前3時に誰かを起こす必要があるのだろうか？

どこに問題があるのかを把握し、目にしている問題が実際に心配する必要があるものなのかを理解することは、可動パーツが増えれば増えるほど、より複雑になる。マイクロサービスアーキテクチャの成長に合わせて、監視とトラブルシューティングの方法を変更する必要がある。これは、常に注意と投資を必要とする領域だ。

5.5.1　この問題はいつ発生するか？

この問題がいつ発生するかを正確に予測するのは少し厄介だ。単純な答えは「本番で何かがうまくいかなくなったとき」ということになる。しかし、どこで何がうまくいかなかったのかを調べることは、本番に入る前に開発者やテスターが対処しなければならないことかもしれない。2、3のサービスしか持っていない段階でこれらの制限にぶつかる可能性もあるが、おそらくサービス数が20以上になるまでは大丈夫だろう。

既存の監視方法がいつ失敗し始めるかを正確に予測するのは難しい。そのため、私が提案できるのは、基本的な改善を前もって実施することを優先することだ。

5.5.2　こうした問題はどのように現れるか？

これは、ある意味ではかなり簡単に見つけられる。説明できない、または理解できない本番環境の問題を目の当たりにしたり、システムが一見健全であるにもかかわらずアラートが発生したり、「全て問題ないか」という単純な質問に答えるのが難しくなるからだ。

5.5.3　取り得る解決策

マイクロサービスアーキテクチャで生じる問題を監視したりトラブルシューティングしたりする方法を変えるのに役立つ多くの仕組みがある。それらの中には実装が簡単なものもあれば、複雑なものもある。そのうちの重要ないくつかを次に示す。

ログ集約

少数のマシン、特に寿命の長いマシンでは、ログをチェックする必要がある場合、通常はマシン自体に入り、そこで情報を取得していた。マイクロサービスアーキテクチャの問題点は、より多くのプロセスがあり、多くのマシン上で実行されていることが多く、短命になることがあることだ（仮想マシンやコンテナなど）。

ログ集約システムを使えば、分散している全てのログをキャプチャして中央へと送り、検索可能にできる。場合によってはアラートを生成するのに使用することも可能だ。オープンソースのELKスタック[†4]から個人的に気に入っているHumio[†5]まで、多くの選択肢が存在するが、これらのシステムは信じられないほど便利だ。

[†4] ログ収集・解析・可視化のためのスタック。Elasticsearch、Logstash、Kibanaの頭文字が取られている。Logstashの代わりにFluentdを用いることも多い。https://www.elastic.co/jp/what-is/elk-stack

---

[†5] https://humio.com

マイクロサービスアーキテクチャを実装する前に、まずログ集約を実装することを強く考えよう。これは信じられないほど便利で、あなたの組織が運用を変化させる能力があるかどうかの良いテストになる。

ログ集約は実装が最も単純な仕組みの1つであり、早い段階で行うべきものだ。実際、マイクロサービスアーキテクチャを実装する際には、最初に行うべきことだろう。これは、最初の段階からとても便利だからだ。さらに、もしあなたの組織が適切なログ集約システムの実装に苦労しているのであれば、マイ

クロサービスの準備ができているかどうかを再考した方がいいかもしれない。ログ集約システムを実装するために必要な作業はかなり簡単で、組織としてその準備ができていないのであれば、マイクロサービスは一歩踏み込みすぎている可能性が高い。

トレーシング

マイクロサービス間の一連の呼び出しがどこで失敗したのか、どのサービスがレイテンシースパイクを起こしたのかを理解するのは、各サービスからの情報だけを分離して分析していては難しいだろう。一連のフローを照合して全体を見られるのは、非常に便利だ。

手始めに、図5-6に示すように、システムに入ってくる全ての呼び出しについて相関IDを生成してみよう。請求書サービスが呼び出しを受信すると、相関IDが与えられる。それが通知サービスへの呼び出しをディスパッチするとき、その相関IDを受け渡す。これはHTTPヘッダ、メッセージペイロードのフィールド、または他のメカニズムを介して行われる。最初の相関IDの生成は、APIゲートウェイやサービスメッシュで行うのが普通だ。

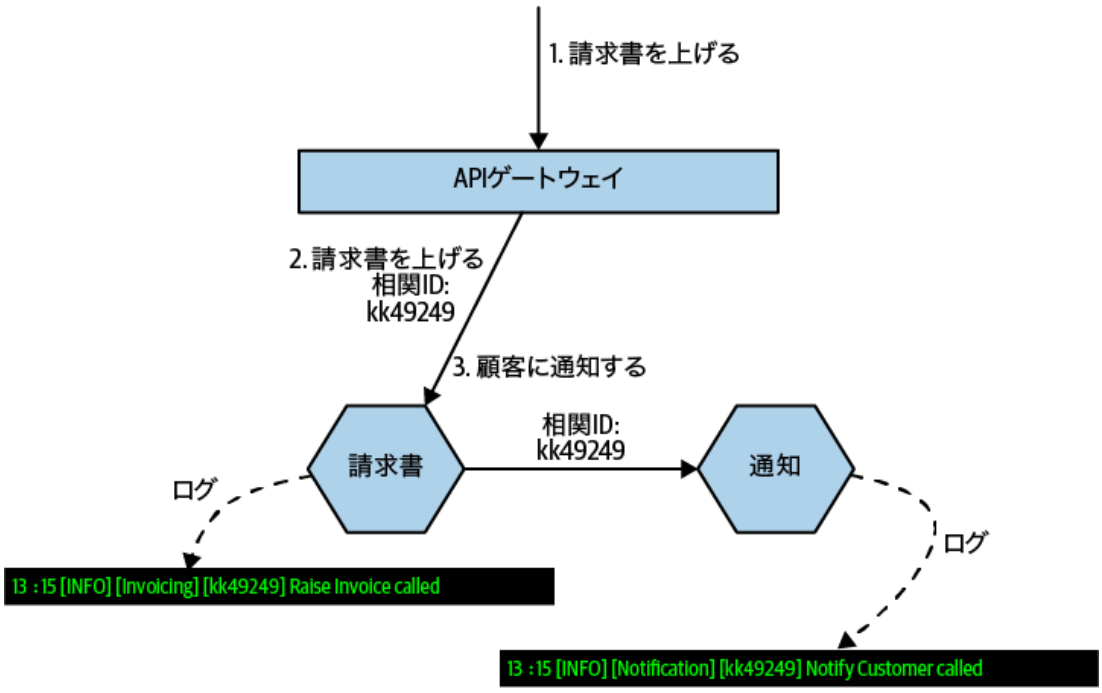

図5-6　相関IDを使用して、特定の呼び出しチェーンに関する情報を収集できるようにする

通知サービスが呼び出しを処理するとき、何をしているかについての情報を同じ相関IDと関連付けてログに記録でき、ログ集約システムを使用して、与えられた相関IDに関連付けられた全てのログを問い合わせられる（ログフォーマッ

トの標準的な場所に相関IDを置くことを前提としている)。もちろん、サーガの管理など、相関IDを使って他のことをすることも可能だ（4章で説明したとおりだ)。

このアイデアをさらに発展させて、通信にかかった時間を追跡するツールを使用することもできる。ログ集約システムの仕組み上、ログは定期的にバッチ化されて中央のエージェントに転送されるため、一連の通信中にどこに時間が費やされているかを判断するための正確な情報を得ることはできない。これには、図5-7に示すオープンソースのJeager[†6]のような、分散トレーシングのシステムが役立つ。

[†6] https://www.jaegertracing.io

アプリケーションがレイテンシーに敏感であればあるほど、私はこうした分散トレーシングツールの実装をすぐに検討するようにしている。もし、相関IDの生成と使用を既存のマイクロサービスアーキテクチャに組み込んでいるのであれば（通常、分散トレーシングツールが必要になる前にそれを行うことを勧めている)、適切なツールにデータをプッシュするよう変更が容易な箇所が、既存のサービススタック中に既にある可能性が高い。サービスメッシュの使用は、個々のマイクロサービス内の呼び出しを計測できなくても、少なくともインバウンドとアウトバウンドのトレースを処理できるので、助けになる。

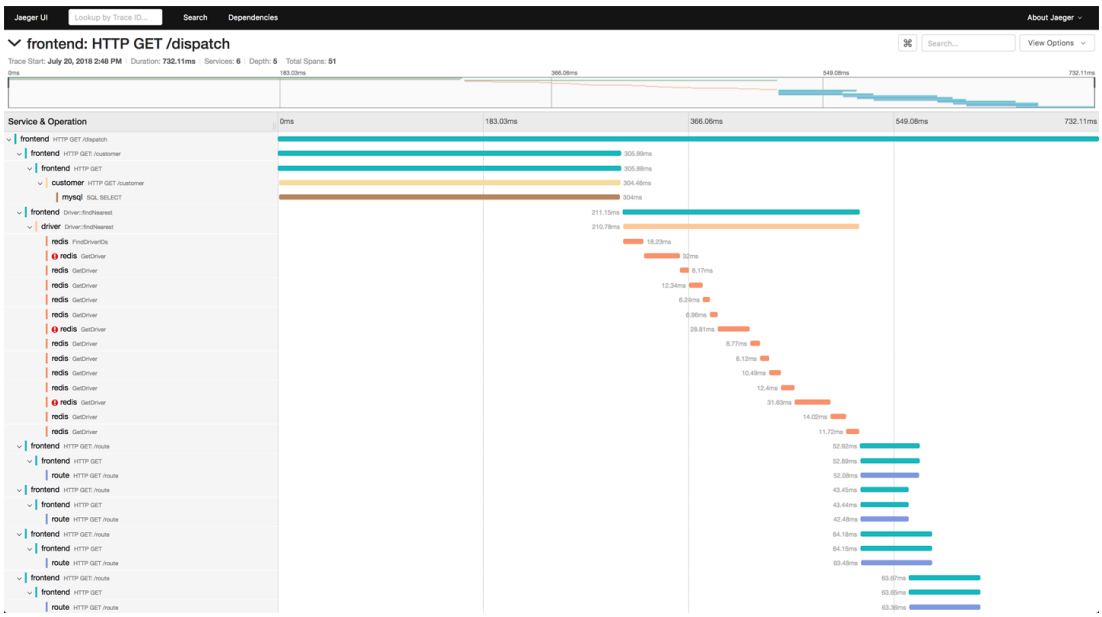

図5-7　Jaegerは、分散トレースの情報を取得し、個々の呼び出しのパフォーマンスを分析するためのオープンソースツールだ

本番環境でのテスト

自動化された機能テストは通常、ソフトウェアがデプロイするのに十分な品質かどうかをフィードバックするためにデプロイ前に行われる。しかし、一度本番環境に導入した後でも、同じようなフィードバックが必要になる。あるフィーチャーが本番環境で一度動作していたとしても、新しいサービスの導入や環境の変化によって、その機能が後になって壊れてしまう可能性があるからだ。

システムに偽のユーザー行動を注入することで、定義された期待する動作に沿わなかったときに、適切なアラートを出すことができる。これはしばしば**人工トランザクション**[†7]と呼ばれる形式で行われる。私が以前勤めていたAtomistという会社では、新規顧客への導入プロセスがやや複雑で、GitHubとSlackの両方のアカウントを使ってソフトウェアを承認する必要があった。このプロセスではさまざまな動きをする必要があり、当初はGitHub APIの呼び出しが制限されるなどの問題が発生するほどだった。そこで、同僚のSylvain Hellegouarchは、偽の顧客を登録するスクリプトを作成した。私たちは定期的に、これらの偽顧客のサインアッププロセスをトリガーにして、プロセス全体をエンドツーエンドでスクリプト化していた。これが失敗するのは、システムに何か問題があることの兆候である場合が多く、それは実際のユーザーが目にするよりも「偽の」ユーザーで発見できる方が断然良い。

[†7] 訳注：スクリプトやツールを用いてサービスの使用をシミュレートする方法。

本番環境でのテストの良い出発点は、既存のエンドツーエンドのテストケースを利用して、それらを本番環境で使用できるように作り直すことだ。考えるべき重要な点は、これらの「テスト」が本番環境で予期せぬ影響を与えないようにすることだ。Atomistでは、合成トランザクションで使用するためのGitHubとSlackのアカウントを管理していたので、本物の人間が巻き込まれたり影響を受けたりすることはなく、スクリプトで簡単に後始末できた。一方で、ある会社では、テスト注文が実際に送信されてしまうという事実を適切に説明していなかったために、誤って200台の洗濯機を本社に注文してしまったという報告を聞いたこともある。本番環境のテストにはくれぐれも気をつけよう。

可観測性に向けて

従来の監視やアラートのプロセスでは、何がうまくいかなくなるかを考え、そうなったときに私たちが分かるよう情報を収集し、それを使ってアラートを出していた。そのため、私たちは主に、ディスク容量の不足、サービスインスタンスが応答しない、レイテンシーの急増といった既知の問題の原因に対処できるよう、自分たちのプロセスを設定していた。

システムが複雑になればなるほど、システムがどんな厄介な方法で私たちを失望させてくるかを全て予測することはますます難しくなってくる。そのときに重要になってくるのは、これらの問題が発生した場合に、システムに対してオープンエンドの質問をできるようにすることだ。それによって、最初に出血を止めてシステムが動作し続けられるようにすることができるし、また将来的に問題を解決するのに十分な情報を収集できるようになる。

そのため、システムが何をしているのかについて多くの情報を収集する必要がある。これにより、調査する必要があると気付いていなかったデータを、後から調査できるようになる。トレーシングとログは、憶測ではなく、実際の情報から問題を判断するための重要なデータソースとなり得る。秘訣は、この情報を簡単に参照し、前後の文脈を追えるようにしておくことだ。

答えを知っていると思い込んではいけない。むしろ、驚かされるようなことがあるのだという考え方をとり、システムに対してうまく質問できるようになり、情報をアドホックに参照できるツールチェーンを使用するようにしよう。もしこの考え方をもっと詳しく調べたいのであれば “Distributed Systems Observability” （O'Reilly）[32]を素晴らしい出発点としてお勧めする。

5.6　開発者体験

サービスが増えれば増えるほど、開発者体験が低下し始める可能性がある。JVMのようなリソース集約型のランタイムでは、1台の開発者用マシンで実行できるマイクロサービスの数が制限される。私のラップトップ上で、4〜5つのJVMベースのマイクロサービスであれば別々のプロセスとして実行できるかもしれない。しかし、10〜20個のマイクロサービスを実行できるだろうか？　おそらく実行できないだろう。負担の少ないランタイムであっても、手元で実行できる数には限りがあり、1台のマシンでシステム全体を実行できない場合はどうすればいいかという会話が必然的に生まれてくる。

5.6.1　この問題はどのようにして現れるか？

立ち上げなければならないサービスが増えるため、手元でのビルドや実行に時間がかかり、日々の開発プロセスが遅くなる可能性がある。開発者は、処理しなければならないサービスの数に対処するために、より強力なマシンを要求し始めるだろう。短期的な解決策としてはそれでいいかもしれないが、サービス資産が増え続けている場合には、それは単に時間を稼ぐことにしかならない。

5.6.2　この問題はいつ発生するか？

この問題がいつ発生するかは、開発者が手元で実行したいサービスの数と、それらのサービスのリソースフットプリントを組み合わせた関数となる可能性が高い。Go、Node、Pythonを使用しているチームは、リソースの制約にぶつかる前に、より多くのサービスを手元で実行できる可能性があるが、JVMを使用しているチームは、この問題に早くぶつかるかもしれない。

また、複数のサービスについて共同所有を実践しているチームは、この問題の影響を受けやすいだろう。開発中に異なるサービス間での切り替え機能を必要とする可能性が高くなる。少数のサービスについて強い所有を行っているチームは、ほとんどの場合、自分たちのサービスだけに集中し、自分たちのコントロール外のサービスをスタブ化するための仕組みを開発する可能性が高くなる。

5.6.3　取り得る解決策

手元で開発を行いたいが実行するサービスの数は減らしたいという場合、一般的な手法は、自分で実行したくないサービスを「スタブ化」するか、他の場所で実行されているインスタンスをサービスとして指定する方法を持つことだ。純粋なリモート開発のセットアップでは、よりキャパシティのあるインフラストラクチャ上でホストされている多くのサービスに対して開発を行える。しかし、そうした方法は接続性が必要となる。これは主にリモートワーカーや出張の多い人にとって問題となるだろう。さらに、動作確認の前にリモート環境へソフトウェアをデプロイする必要があるためにフィードバックサイクルが遅くなったり、開発環境に必要なリソース（とそれに伴うコスト）が爆発的に増える可能性がある。

Telepresence[†8]は、Kubernetesユーザーがローカル／リモートのハイブリッドな開発ワークフローをより簡単に行えることを目的としたツールの一例だ。

サービスはローカル環境で開発しつつ、Telepresenceが他のサービスへの呼び出しをリモートクラスタにプロキシする。これによって（うまくいけば）両方の世界を最大限に利用して開発できる。また、Azure Functionsは、リモートのクラウドリソースに接続されているがローカル環境でも実行できる。そのため、広範囲のクラウド環境に対して実行を行いながら、ローカル環境での高速なワークフローと共にAzure Functionsを利用したサービスを開発できる。

[†8] https://www.telepresence.io

サービスの数が増えるにつれて開発者体験がどのように変化するかを確認することは重要であり、そのためにはフィードバック機構を設置する必要がある。サービス数が増加していく中で開発者が生産的であり続けられるようにするためには、継続的な投資が必要になる。

5.7　あまりにも多くのものを実行している

サービス数が増え、それらのサービスインスタンスが増えれば、デプロイ、設定、管理する必要のあるプロセスも増える。モノリシックアプリケーションのデプロイと設定を処理するための既存の技術は、管理する必要のある可動パーツの数が増えると、うまくスケールできなくなる可能性がある。

特に、望ましい状態の管理がますます重要になってくる。**望ましい状態の管理**とは、必要なサービスインスタンスの数と場所を指定し、これが時間の経過とともに維持されることを保証する能力だ。現在はモノリスで手動プロセスを使用して管理しているかもしれないが、マイクロサービスが何十、何百もある場合、特にそれぞれのマイクロサービスが異なる望ましい状態を持っている場合には、それはうまく拡張できない。

5.7.1　この問題はどのようにして現れるか？

デプロイの管理とデプロイ中に発生する問題のトラブルシュートに費やされる時間の割合が増えてくる。プロセスが手作業に依存している場合、間違いは必ず起こる。

サービスやサービスインスタンスの数が増えるにつれ、本番環境のデプロイと保守に関連した活動を管理するために、より多くの人員が必要になることに気

付くだろう。その結果、運用チームをサポートするための人員の増員を要求されたり、デリバリーチームがデプロイの問題に費やす時間の割合が高くなったりする可能性がある。

5.7.2　この問題はいつ発生するか？

この問題は全てスケールに関係している。マイクロサービスが増え、それらのマイクロサービスのインスタンスが増えれば増えるほど、手動プロセスやChefやPuppetのような従来の自動化された構成管理ツールはもはやあてはまらなくなる。

5.7.3　取り得る解決策

この問題を解決するには、高度な自動化を可能にし、理想的には開発者がセルフサービスでデプロイ環境をプロビジョニングでき、自動化された望ましい状態の管理を処理できるツールが必要となる。

マイクロサービスでは、Kubernetesがまさにこの分野のためのツールとして登場している。サービスをコンテナ化する必要があるものの、一度コンテナ化してしまえば、Kubernetesを使って複数のマシンにまたがるサービスインスタンスのデプロイを管理でき、堅牢性を向上させて負荷を処理するための拡張性を確保できる（十分なハードウェアを持っていることが前提だ）。

素のKubernetesが開発者にとって使いやすいものだとは思っていない。より高次で開発者に優しい抽象化に多くの人が取り組んでおり、私はその作業が続くことを期待している。将来的に、Kubernetesが実装の詳細に過ぎなくなるにつれて、Kubernetes上でソフトウェアを実行している多くの開発者は、そのことを意識しなくなるだろう。大規模な組織では、RedHatのOpenShiftのようなKubernetesのパッケージ版を採用する傾向があるが、これはKubernetesを企業環境内での作業を容易にするツールとバンドルしたもので、企業でよく使われている認証方式やアクセス管理制御などを扱えるようにしたものだ。これらのパッケージ版の中には、開発者が作業するための簡略化された抽象化を提供するものもある。

運よくパブリッククラウドを利用している場合は、マネージドなKubernetesを含むマイクロサービスアーキテクチャのデプロイを処理するために、さまざまなオプションを利用できる。たとえば、AWSとAzureは、この領域で複数の選択肢を提供している。私はFaaS（Function-as-a-Service）の大ファンだ。

これは、いわゆる**サーバーレス**と呼ばれるもののサブセットだ。適切なプラットフォームがあれば、開発者はコードを気にするだけで、基礎となるプラットフォームが運用作業のほとんどを処理してくれる。現在提供されているFaaSには限界があるが、それにもかかわらず、運用上のオーバーヘッドを大幅に削減できる見通しを提供している。

既にパブリッククラウドを利用しているチームの場合、私はKubernetesや同様のコンテナベースのプラットフォームからは始めないことが多い。その代わり、運用作業の軽減を理由に、「サーバーレスファースト」というアプローチを採用している。「サーバーレスファースト」とは、FaaSのようなサーバーレス技術を利用するのをまず試みるアプローチだ。もしあなたの問題が、利用可能なサーバーレス製品の制限に合わないのであれば、他の選択肢を探してほしい。明らかに全ての問題空間が同じというわけではないが、既にパブリッククラウドを利用しているのであれば、Kubernetesのようなコンテナベースのプラットフォームの複雑さは常に必要なものではないと感じている。

マイクロサービスを採用するプロセスにおいて、それが前提条件だと思い込んで、早い段階でKubernetesなどに手を伸ばす人々をよく目にする。しかし、それらは前提ではない。たしかに、Kubernetesのようなプラットフォームは複数のプロセスを管理するのに優れている。しかし、現在のアプローチや技術に負担がかかり始めるほどにプロセスが増えるまでは、それに手を出すのは待つべきだ。必要なのは5つのマイクロサービスだけかもしれないし、既存のやり方のままでこれをうまく処理できるかもしれない。他の人がやっているからといって、Kubernetesベースのプラットフォームを採用してはいけない。もちろん、これはマイクロサービスにも言えることだ。

5.8　エンドツーエンドテスト

自動化された機能テストは何であれ、微妙なバランス調整を必要とする。テストする機能が多ければ多いほど、テストの範囲が広ければ広いほど、アプリケーションへの信頼は高まる。一方で、テストの範囲が広ければ広いほど、実行に時間がかかり、失敗したときに何が壊れているのかを突き止めるのが難しくなる。

どんな種類のシステムであっても、エンドツーエンドのテストは、スコープがより小さなユニットテストと比べてカバーする機能がずっと多く、テストの記述や保守により多くの問題があることが知られている。しかし、ユーザーと同じようにシステムを使用することで得られるエンドツーエンドテストの信頼は、多くの場合、それだけの価値があるものだ。

しかし、マイクロサービスアーキテクチャでは、エンドツーエンドテストの「スコープ」は非常に大きくなる。複数のサービスにまたがってテストを実行しなければならなくなり、全てのサービスをデプロイし、テストシナリオに合わせて適切に設定する必要がある。また、サービスインスタンスの停止やデプロイの失敗によるネットワークタイムアウトといった、環境の問題が原因でテストが失敗した場合に起こる検出漏れにも備えなければならない。つまり、マイクロサービスアーキテクチャに対するエンドツーエンドテストは、標準的なモノリシックアーキテクチャに対するエンドツーエンドテストよりも、制御外の問題に対してはるかに脆弱だと言える。

そのため、テストの範囲が増えると、発生した問題との戦いに多くの時間を費やすことになり、エンドツーエンドテストを作成して維持しようとすることが膨大な時間の浪費になる。

5.8.1　この問題はどのようにして現れるか？

一つの兆候は、エンドツーエンドのテストスイートが大きくなり、完了するまでに時間がかかるようになることだ。これは、複数のチームがどのシナリオがカバーされているか分からず、「念のため」に新しいシナリオを追加していることが原因だ。エンドツーエンドのテストスイートでは、コードの問題点を浮き彫りにしてくれないようなテストの失敗が多く見られる。そのため、開発者はテストが合格するかどうかを確認するために再度テストを実行することがよくある。

エンドツーエンドのテストに費やされている時間はどんどん長くなる。そのため、テスターの数を増やしたり、別のテストチームを編成したりする必要が出てくる。

5.8.2　この問題はいつ発生するか？

この問題は常に忍び寄ってくる傾向があるものだが、複数のチームがそれぞれ異なるユーザー向けの行動プロセスを処理している状況で最もこの課題が発生しやすいと感じる。各チームの作業が分離していればいるほど、手元でのテスト管理は容易になる。しかし、チームをまたいだフローをテストする必要があればあるほど、エンドツーエンドの大きなスコープのテストがより問題になる。

5.8.3　取り得る解決策

『マイクロサービスアーキテクチャ』[25]では、テストの扱い方を変える選択肢について説明すると共に、そのための章も設けている。ここでは、その内容について簡単に説明する。

自動化された機能テストの範囲を制限する

複数のサービスをカバーするテストケースを書く場合、これらのテストがサービスを管理するチーム内で行われるようにしよう。つまり、チームを横断するような大きなスコープのテストは避けることだ。テストの所有権を単一のチーム内で維持することで、どのようなシナリオが適切にカバーされているかを理解しやすくなり、開発者がテストを実行してデバッグできるようになり、誰がテストを実行して合格させるべきかの責任をより明確にできるようになる。

コンシューマー駆動契約を使う

サービスをまたいだテストケースの必要性を代替するために、コンシューマー駆動契約（Consumer-Driven Contracts：CDC）の使用を考える手がある。CDCでは、マイクロサービスのコンシューマーに、サービスがどのように動作するかについての期待値を実行可能な仕様、つまりテストの観点から定義してもらう。そして、サービスを変更しても、これらのテストが合格していることを保証するようにする。

これらのテストはコンシューマーの視点から定義されているため、偶発的な契約違反を拾い上げることが可能だ。また、コンシューマーの視点から要求を理

解でき、何よりも、それぞれのコンシューマーがいかに多種多様な期待をサービスに持っているかを理解できる。

CDCは単純な開発ワークフローを使用することで実装することもできるが、この技術をサポートするように設計されたツールを使用することで、より簡単にできる。最良の例は、おそらくPact[†9]だろう。

[†9] https://pact.io

このアプローチで大成功を収めたチームもあるし、うまく取り入れることができなかったチームもあるということには注意が必要だ。アイデアは正しいし、うまく機能し得ることもわかっている。しかし、この手法を取り入れる際に一部の人々が抱える課題について、私自身はまだ完全には理解できていない。この方法は、本当に難しい問題を解決するための、まだあまり活用されていないプラクティスだ。

自動化されたリリース修正とプログレッシブデリバリーを使用する

自動テストでは、通常、本番環境に影響が生じる前に問題を見つけようとする。しかし、システムが複雑になるにつれて、これを行うことがますます難しくなることがある。そのため、問題が発生したときに本番環境に与える影響が小さくなるよう努力する価値がある。

3章で触れたように、**プログレッシブデリバリー**とは、顧客にソフトウェアの新バージョンを段階的に公開する手法の総称だ。これは、より小さな顧客グループで新しいリリースの影響を評価することで、リリースを継続するか、ロールバックするかを決定するという考え方だ。プログレッシブデリバリー手法の例としては、カナリアリリースがある。

サービスの振る舞いが満たすべき許容可能な基準を定義することで、自動化された方法でプログレッシブデリバリーを制御できるようになる。単純な例として、95パーセンタイルのレイテンシーとエラー率の許容可能なしきい値を定義し、これらの測定値が満たされた場合にのみ公開を継続することが考えられる。測定値が満たされなかった場合には、最新のリリースを自動的にロールバックして、何が起こったのかを分析する時間を確保することが可能だ。

多くの組織では、このような自動化されたリリース修正技術を使用している。特にNetflixは、このアイデアの使用について長く語ってきている。Netflixは、サービスのプログレッシブデリバリーを制御するデプロイ管理ツールとしてSpinnakerを開発したが、これらのアイデアを実践する方法は他にもたくさんある。

テストの代わりに自動化されたリリース修正を検討すべきだと言っているわけではない。ただ、努力に対して最大限のリターンを得られるのはどこかをよく考えるべきだと言っているだけだ。そもそも問題が発生しないようにすることだけに焦点を当てるのではなく、問題が発生した場合にそれをキャッチすることにある程度の労力をかけることで、はるかに堅牢なシステムが手に入る可能性がある。

これらのテクニックは組み合わせることでうまく機能するが、たとえ自動化されたリリース修正は今すぐにはできないとしても、何らかの形でプログレッシブデリバリーを導入するだけでも大きな価値があると覚えておくことが重要だ。プログレッシブデリバリーを手動で制御するだけでも、ただ単に新しいソフトウェアを全員に公開している状態からの大きなステップアップになる。

品質フィードバックサイクルを継続的に改善する

どこでどのようにテストすべきかを理解することは、継続的な課題だ。アプリケーションをテストする場所と方法を調整するために、開発プロセス全体を総合的に見られる人材が必要となる。ここで必要なのは、運用上の欠陥が増加しているシステム領域をカバーするために新しいテストを追加する必要性を特定できるだけでなく、既にカバレッジがあるときにはフィードバックサイクルを改善するためにテストを削除することもできるような人材だ。

要するに、迅速なフィードバックの必要性と安全性のバランスをとることだ。新しいテストを追加するのと同じように、間違ったテストを特定し、削除したり置き換えたりすることにも積極的でなければならない。

5.9　全体最適と局所最適

チームが局所的な意思決定に対してより多くの責任を持ち、管理するマイクロサービスのライフサイクル全体を所有しているというモデルを受け入れると、局所的な意思決定とより全体的な関心ごととのバランスを取る必要が生じる段階に到達するだろう。

この問題がどのように顕在化するかの例として、請求書、通知、フルフィルメントサービスを管理する3つのチームを考えてみよう。請求書チームは、Oracleをよく知っているのでデータベースに Oracleを使うことにした。通知チームはMongoDBを使いたいと考えている。一方、フルフィルメントチームは、既にPostgreSQLを使っているため、PostgreSQLを使いたいと考えている。それぞれの決定を順番に見ていくと、それは完璧に理にかなっており、そのチームがどのようにしてその選択をしたかを理解できる。

しかし、一歩下がって全体を見てみると、組織として、多少似たような機能を持つ3つのデータベースのためにスキルを構築し、ライセンス料を支払いたいかどうかを自問しなければならない。全ての人にとって完璧ではないが、ほとんどの人にとっては十分なデータベースであることを受け入れて、1つのデータベースだけを採用した方が良いのだろうか？　局所的に起こっていることを、全体的な文脈に置いて考える力無しに、どうやってこうした種類の決定を下せるのだろうか？

5.9.1　この問題はどのようにして現れるか？

この問題を目の当たりにする最も一般的な機会は、複数のチームが意識しないままに同じ問題を異なる方法で解決していたということに、誰かが突然気付いたときだ。時間がたつにつれて、これは信じられないほど非効率的になることがある。

オーストラリアの不動産会社REAの人々と話したことを覚えている。彼らは、何年もマイクロサービスを構築しているうちに、チームがサービスをデプロイする方法がたくさんあることに気付いた。これは、人々があるチームから別のチームに移ったときに、新しいやり方を学ばなければならないという問題を引き起こした。また、各チームが行っていた重複した作業を正当化することも難しくなった。その結果、彼らはこれを処理するための共通の方法を考え出すために、いくつかの作業を行うことにした。

このような問題は、たいていの場合、ランチの時に偶然耳にした一言がきっかけで気付くことが多い。コミュニティ・オブ・プラクティス（実践コミュニティ）のような、ある種のチーム横断的な技術グループがあれば、これらの問題をはるかに早く発見できる。

5.9.2　この問題はいつ発生するか？

この問題は、複数のチームから成る組織では時間の経過とともに発生する傾向がある。特に、チームがどのように仕事を進めていくかについて、より自由度が高くなっている組織では特に発生する問題となる。マイクロサービスの旅の初期段階では、この問題のことは気にしなくてよい。おそらく、物事がどのように行われるべきかについて明確な共通理解を持ってスタートすることだろう。時間がたつにつれて、各チームはそれぞれの局所的な問題にますます焦点を当て、問題を解決する方法をそれに基づいて最適化するようになる。そうすると、「これが私たちのやり方だ」というコアな共通見解が変化し始めるだろう。

組織の規模拡大期を経た後に、この問題が議論されるようになるのをよく目にする。短期間に多くの開発者が流入すると、アドホックに共有された情報がスケールしにくくなるからだ。これにより、情報のサイロ化が進み、橋渡しが必要になる可能性がある。

サービスの共同所有を実践していれば、この問題を回避するか、少なくとも限定的にできるだろう。サービスを共同所有するには、問題をどう解決するかについてのある程度の一貫性が求められるからだ。別の言い方をすれば、共同所有を望むならば、この問題を解決しなければならない。

5.9.3　取り得る解決策

この領域に役立ついくつかのアイデアについては既に触れた。2章では、図5-8であらためて示すように、不可逆的な判断と可逆的な判断という考え方を探った。変化のコストが高ければ高いほどインパクトが大きく、意思決定の背後にはより広いコンセンサスが必要になる。インパクトが小さければ小さいほどロールバックが容易になり、意思決定をローカルチームに任せられるようになる。

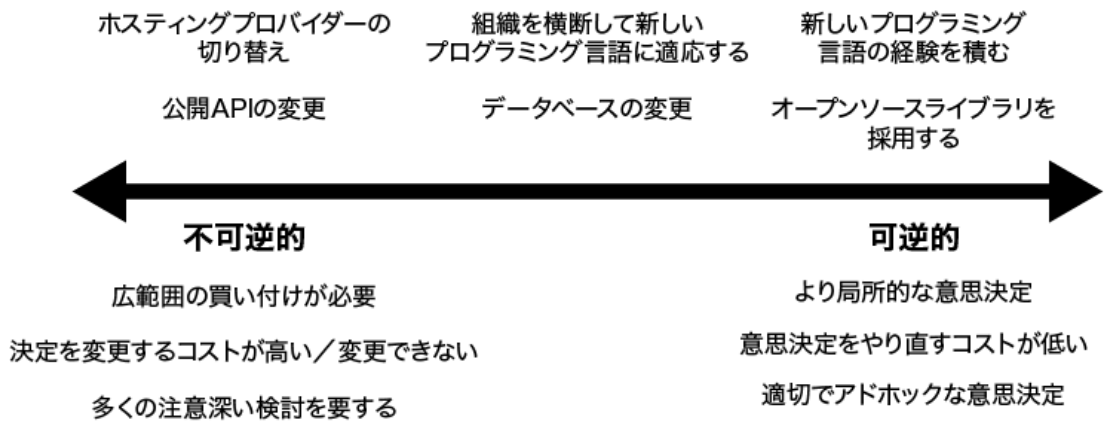

図5-8　スペクトルに沿った例で示す不可逆的決定と可逆的決定の違い

自分たちの意思決定が不可逆的なものと可逆的なもののどちらに傾いているかを、チームの人々が認識できるようにすることが秘訣だ。意思決定が不可逆的な方向に傾いているほど、チームの境界外の人を意思決定に参加させることが重要になる。これをうまく機能させるためには、チームは少なくとも大局的な関心ごとへの基本的な理解を持ち、それと自分たちの関心ごとが重なりそうな領域を見極める必要がある。また、これらの問題を表面化させ、他チームの同僚の関与を得られるようなネットワークも必要だ。

簡単な仕組みとしては、各チームから少なくとも1人の技術リーダーを技術横断グループの一員とすることで、こうした懸念に対処できる。CTO、チーフアーキテクト、または会社の全体的な技術的ビジョンの責任者が、このグループのリーダーを務めるのも良いだろう。

この横断的なグループは、チームがより大きなフォーラムで議論したいと欲している局所的な問題を表面化させる場であるだけでなく、横断的な問題を拾い上げる場でもある。複数のチームがそれぞれ異なる方法で問題を解決していること、そしてそれらは全体レベルで解決する方が賢明であるということに、チーム間のコミュニケーション抜きでどうやって気付くのだろうか？

組織の性質によっては、その場限りの非公式なプロセスに頼れないかもしれない。たとえば、Monzoでは、社内で「提案」と呼ばれるフリーフォームの文書を提出できるようにしている。提案は共有スペースに公開され、新しい提案が見られるようになったことは、Slackを通じて全社に通知される。ステークホルダーはその提案について議論したり、提案に磨きをかけたりできる。これらの提案は完成品ではなく、むしろ変更に開かれているべきことが期待されている。これはMonzoにとってはうまくいっているようだが、その理由の一つには、コミュニケーションと責任の共有という文化がある。

基本的に、各組織は全体的な意思決定と局所的な意思決定の間に適切なバランスを見つける必要がある。どの程度の責任をチームに渡せたら満足できるだろうか？　また、中央でどの程度の管理をしたいと考えているだろうか？　チームに責任を渡せば渡すほど、自律性が高まるというメリットがあるものの、問題解決の方法に一貫性がなくなるというトレードオフがある。中央から物事を推進すればするほど、合意形成の必要性が高まり、それがペースを落とすことになるだろう。この2つの力のバランスをどうやって取るのがあなたにとって正解かを、私が教えることはできない。あなたは自力で考えなければならない。このバランスが存在することを認識し、適切な情報を収集して、時間をかけてバランスを調整できるようにする必要がある。

5.10　堅牢性と回復性

分散システムは、モノリシックなシステムに慣れている人が見慣れないさまざまな障害モードを示すことがある。ネットワークパケットが失われたり、ネットワーク呼び出しがタイムアウトしたり、マシンが死んだり応答しなくなったりすることがある。このような状況は、従来のモノリシックアプリケーションのような単純な分散システムでは滅多に起きないことかもしれないが、サービスの数が増えるにつれて、これらは当たり前になってくる。

5.10.1　この問題はどのようにして現れるか？

これらの問題は、残念ながら、本番環境で発生する可能性が高い。従来の開発とテストのサイクルでは、短期間だけ本番環境のような状況を再現している。このような稀な問題は発生する可能性が低く、発生したとしても、しばしば無視されてしまう。

5.10.2　この問題はいつ発生するか？

もしあなたのシステムがいつ不安定性に苦しむかを事前に教えられるとしたら、私はこの本を書いていない。おそらくどこかのビーチでモヒートを飲みながら時間を過ごしているだろう。私が言えることは、サービスの数が増え、サービス呼び出しの数が増えれば増えるほど、回復性の問題に対してますます脆弱になるということだ。サービスが相互に接続されていればいるほど、カスケード障害やバックプレッシャー[†10]などの課題に悩まされる可能性が高くなる。

[†10] 訳注：受信側の処理が追いつかない場合などに、受信側から送信側に対してフロー制御を行えるようにすること。

5.10.3　取り得る解決策

良い出発点は、それぞれのサービス呼び出しに関して、次の2つの質問に答えてみることだ。この呼び出しがどのように失敗し得るかを分かっているだろうか？　そして、呼び出しが失敗した場合、何をすべきかを分かっているだろうか？

これらの質問に答えられれば、多くの解決策を考え始められる。一時的結合（1章参照）を避けるために非同期通信を導入するなど、サービス同士を相互に分離することは、解決の助けとなるだろう。タイムアウトを賢く行えば、遅い下流サービスがリソース競合を起こしてしまうのを避けられるし、サーキットブレーカーのようなパターンと組み合わせれば、早期に失敗させることでバックプレッシャーを避けることもできる。

サービスの複数のコピーを稼働することは、（クラッシュした場合にサービスが再起動されるような）望ましい状態の管理を実装できるプラットフォームと同様に、インスタンスの死滅を防ぐ助けとなり得る。

2章でも述べたが、回復性とは単にいくつかのパターンを実行するだけのものではない。回復性とは、予測できない問題（これが思いがけなく発生することは避けられない）が発生した場合にそれに対処できるだけでなく、必要に応じて仕事の仕方を進化させることもできる組織を構築すること、つまり、働き方全体に関わるものなのだ。この考えを実践する具体的な方法としては、本番環境上の問題が発生したときにはそれを文書化し、学んだことを記録しておく方法が挙げられる。ひとたび問題を解決するか迂回するかした後に、そこから学ぶことなくさっさと次へと進んでいき、数カ月たって同じ問題に再び出くわしているという組織をあまりに多く目にする。

正直言って、ここではまだ表面をさらった程度だ。これらの考え方をより詳しく調べるには、『マイクロサービスアーキテクチャ』[25]の11章を読むか、Michael Nygardによる『Release it!』（オーム社）[26]を読むことをお勧めする。

5.11　孤児サービス

妙な話だが、素晴らしい技術を持ち、信じられないほど複雑でスケーラブルなシステムを構築しているにもかかわらず、私たちは平凡な問題にも引き続き頭を悩ます。そうした問題の一つとして、たとえば、多くの組織が、自分たちが何を持っているのか、どこにあるのか、誰が所有しているのかを正確に把握することに苦労しているのを目にする。

マイクロサービスがその目的にますます焦点を当てるようになるにつれ、数週間、数カ月、あるいは数年に渡って何の変更も加えられることなく快調に稼働しているサービスが増えてくる。ある面において、これはまさに私たちが求めていたことだ。システムの残りの部分を安定させられるからこそ、独立デプロイ可能性は魅力的な概念なのであり、システムの中で変更が必要ない部分の安定性を維持するのは良い考えだからだ。

基本的に組織の誰もそれらの所有権や責任を負っていないため、私はこれらのサービスを**孤児サービス**と呼んでいる。

5.11.1　この問題はどのようにして現れるか？

古いオフィスで壁に囲まれた古いサーバー群が発見されたという話を聞いたことがある。誰もそのサーバー群がそこにあることを覚えていなかった。そして、何をしているかはともかく、そのサーバー群は快調に稼働し続けていた。新たに発見されたそれらのコンピューターが何をしているのかを正確に覚えている人は誰もいなかったため、怖くて誰もその電源を切りたがらなかった。マイクロサービスも同じような特徴を示す可能性がある。マイクロサービスはそこにあり、動いている（と私たちは思い込んでいる）のだが、それをどうしたらよいのか分からないという同じ問題がある。その恐れが、サービスを変えることを躊躇させてしまうことがある。

根本的な問題は、もしこのサービスが機能しなくなったり、変更が必要になった場合、人々は何をすべきか分からず途方に暮れてしまうということにある。問題のサービスのソースコードがどこにあるのか分からないという話をしている複数のチームと話したことがあるが、そうしたことはかなり大きな問題だ。

5.11.2　この問題はいつ発生するか？

この問題は通常、マイクロサービスを長期間にわたって使用してきた組織で発生する。このマイクロサービスに関わっている人たちは、そのサービスでしたことを忘れてしまったか、おそらく会社を辞めてしまったのだろう。

5.11.3　取り得る解決策

検証したわけではないものの、サービスの共同所有を実践している組織はこの問題を起こしにくいのではないかということを私は仮説として持っている。この種の組織は、サービス間のコンテキスト切り替えのコストを減らすために、言語や技術の選択を既に制限しているはずだ。また、サービスに変更を加えたり、テストしたり、デプロイしたりするための共通のツールを持っている可能性もある。サービスが最後に変更された後でそうした共通のプラクティスが変更されている場合には、もちろん、これは助けにはならないかもしれない。

このような問題を抱える多くの企業と話をしてきたが、最終的にシンプルな社内レジストリを作成することが、サービスに関するメタデータ照合の助けとなることが多かった。これらのレジストリの中には、単にソースコードリポジトリをクロールして、そこにあるサービスのリストを構築するためにメタデータファイルを探しているものもある。この情報をconsulやetcdのようなサービスディスカバリーから得られた実際のデータと突き合わせることで、何が実行されているのか、誰に相談できるのかのより豊かなイメージを構築できる。

フィナンシャル・タイムズ紙は、この問題に対処するためにBiz Opsを作った。同社は、世界中のチームが開発した数百のサービスを所有している。Biz Opsツール（図5-9）は、ネットワークやファイルサーバーなどのITインフラサービスに関する情報に加えて、同社のマイクロサービスに関する多くの有用な情報を1箇所で見つけられるようにしてくれるものだ。グラフデータベース上に構築されているため、どのようなデータを収集し、どのように情報をモデル化するかについて、多くの柔軟性を持っている。

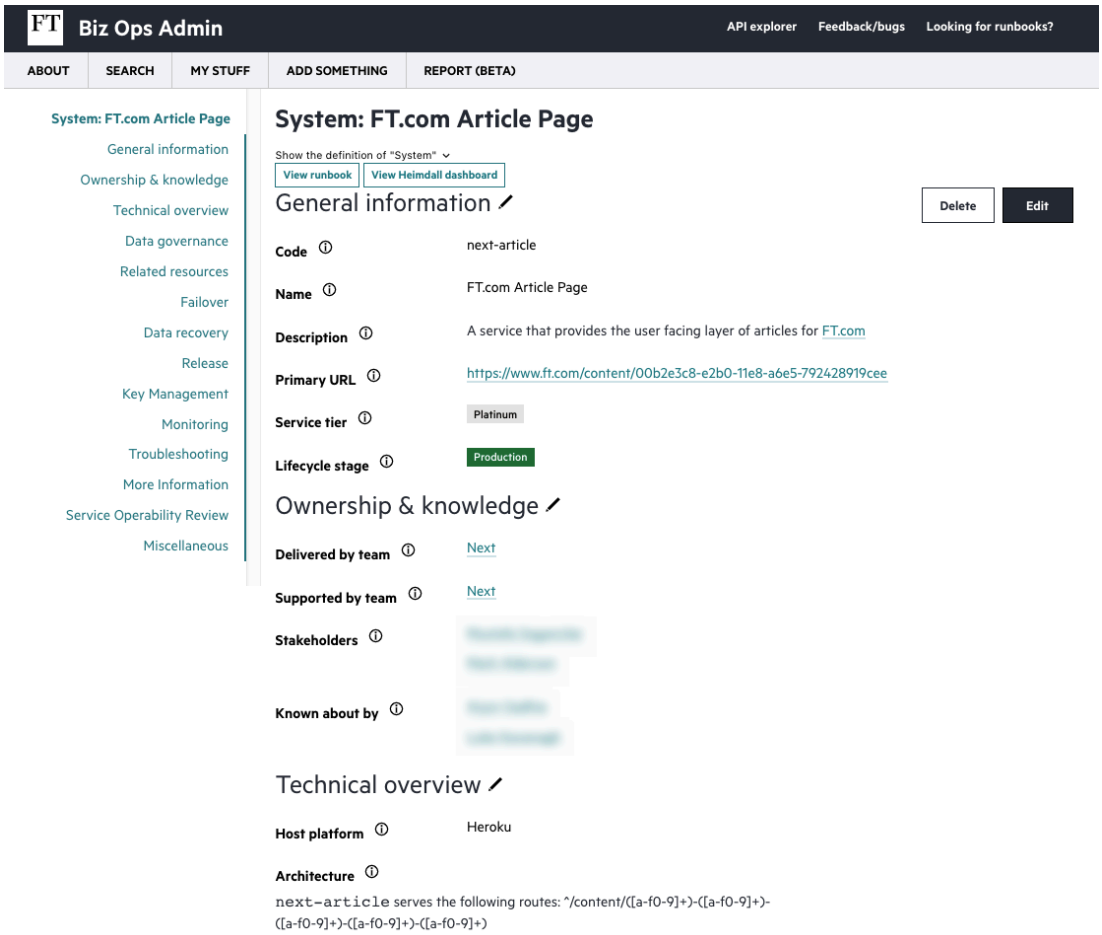

図5-9　マイクロサービスに関する情報を収集するフィナンシャル・タイムズ紙のBiz Opsツール

そればかりか、Biz Opsツールは、私がこれまで見てきたほとんどのツールよりも、ずっと先を行っている。図5-10に示すように、システム操作性スコアと呼ばれるものを計算する。サービスを運用しやすくするには、サービスとそのチームが実行すべき特定の事柄があるというのが、システム操作性スコアの考え方だ。システム操作性スコアには、チームがレジストリに正しい情報を提供したことを確認することから、サービスに適切なヘルスチェックが行われることを確認することまで含まれる。これらの計算されたスコアによって、チームは修正が必要なものがあるかどうかを一目で確認できるようになっている。

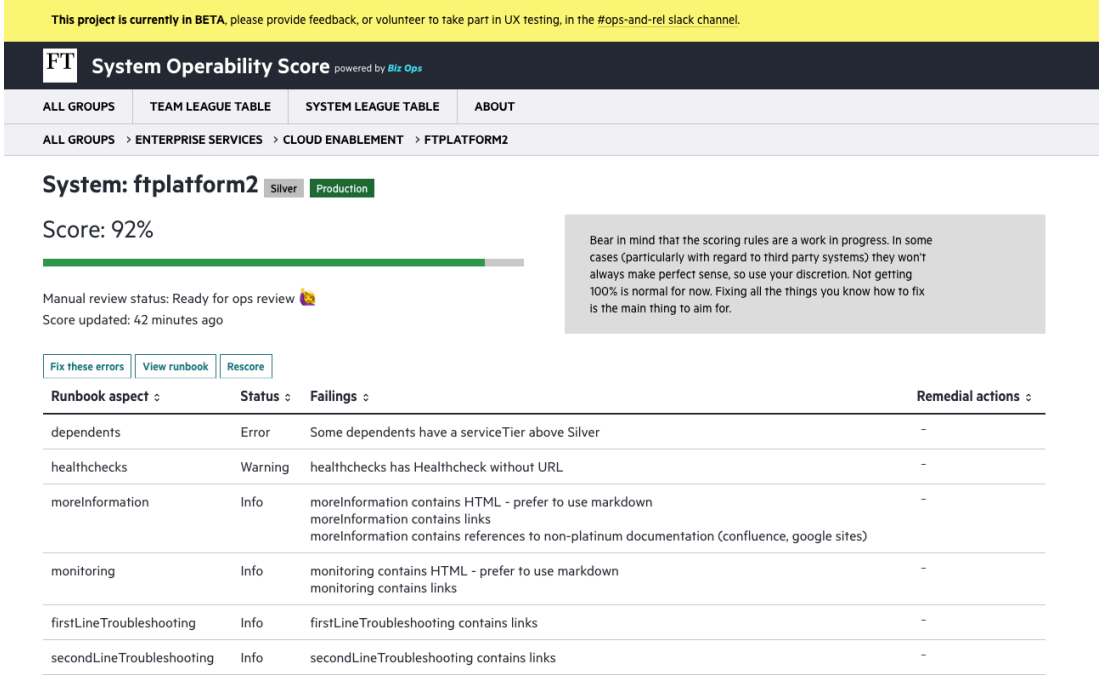

図5-10　フィナンシャル・タイムズ紙のマイクロサービスにおけるシステム操作性スコアの例

サービスレジストリのようなものを持つことは助けになるが、孤児サービスがレジストリよりも前にあった場合はどうなるだろうか？　重要なのは、新たに発見された「孤児化した」サービスを、他のサービスの管理方法と一致させることだ。そのためには、既存のチームに所有権を割り当てるか（強い所有権を実践している場合）、サービスを改善するための作業項目を挙げる必要がある（共同所有を実践している場合）。

5.12　この章のまとめ

この章では、マイクロサービスが引き起こす可能性のある全ての問題を網羅的にリストアップしたわけではないし、取り得る解決策を全てリストアップしたわけでもない。代わりに、人々が苦労しているように見える最も一般的な問題に焦点を当てている。

これらの問題がいつ発生するのかについて決まった規則はないということを明確にしておきたい。全ての状況は異なり、多くの要因が関係している。私が試みたのは、未来を見通すことはできないが、少なくとも予見はできるということを伝えることだ。もっと難しいのは、問題が起こる前に問題を解決することと、決して起こらないであろう問題を解決することに時間を費やすこととの間で、適切なバランスを見つけることだ。どんな警告に注意を払うべきかを理解して、この章を後にしてもらえることを願っている。